Learning outcomes:

Để thi:

- Cho một đoạn mã (giả), biết cách tính xấp xỉ thời gian chạy của thuật toán theo số lần chạy một phép toán nào đó (truy nhập mảng, so sánh…)
- Biết phân loại thuật toán theo đánh giá tiệm cận (asymptotic analysis) về tốc độ tăng của thời gian chạy (hằng số, tuyến tính, lô ga rit, n log n, bậc hai…. lũy thừa), trong trường hợp tốt nhất, tồi nhất, trung bình (cái này thì sẽ yêu cầu cụ thể cho từng thuật toán sau này sẽ học trong chương trình)
- Cho một mô tả cấu trúc dữ liệu và thuật toán, đánh giá về bộ nhớ mà thuật toán sử dụng (hằng số, tuyến tính, lô ga rit, n log n, bậc hai…)
- Biết phân biệt Big Oh, Big Theta, Big Omega

Bài giảng

Nếu chỉ học qua qua cơ bản thì dùng bài giảng COS226 mới của Princeton (không nói đến Big Oh (O), Big Omega (Ω) và phân loại độ khó của bài toán)
https://www.cs.princeton.edu/courses/archive/spring26/cos226/lectures/14AnalysisOfAlgorithms.pdf
Nếu muốn học sâu hơn thì dùng slide gốc của giáo trình
https://algs4.cs.princeton.edu/lectures/keynote/14AnalysisOfAlgorithms.pdf

Hướng dẫn học

https://www.cs.princeton.edu/courses/archive/spring16/cos226/lectures/study/14AnalysisOfAlgorithms.html

Ký hiệu xấp xỉ ~ (Tilde notation)

Chúng ta nói rằng f(N) ~ g(N) , đọc là f(N) xấp xỉ g(N) nếu tỉ số f(N)/g(N) tiến về 1 khi N tiến ra vô cùng. Đây là một khái niệm toán học tổng quát áp dụng cho các hàm số và không bị giới hạn riêng trong thời gian chạy, bộ nhớ, hay bất kỳ phạm vi cụ thể nào khác.

Giả thuyết định luật lũy thừa (Power-law assumption)

Đối với các phân tích thực nghiệm trong môn học, chúng ta thường giả định rằng thời gian chạy tuân theo định luật lũy thừa: $T(N) ~ aN^b. Việc suy luận thực nghiệm để đưa ra các giả thuyết chính xác cho những hàm thời gian chạy không tuân theo định luật lũy thừa nằm ngoài phạm vi của khóa học này.

Mô hình chi phí (Cost model)

Đối với các phân tích lý thuyết về thời gian chạy, chúng ta sẽ giả định một *mô hình chi phí*, cụ thể là có một (hoặc vài) thao tác nhất định sẽ chiếm phần lớn thời gian chạy của chương trình, ví dụ phép so sánh hay phép truy nhập mảng. Sau đó, chúng ta biểu diễn thời gian chạy dựa trên tổng số lần thao tác đó được thực hiện dưới dạng một hàm số theo kích thước đầu vào. Để đơn giản hóa, chúng ta thường biểu diễn tần suất đếm này bằng ký hiệu ~.

Tốc độ tăng trưởng (Order of growth)

Nếu chúng ta có hai hàm số f(N) và g(N), và f(N) ~ c g(N) với hằng số c > 0, chúng ta nói tốc độ tăng trưởng của f(N) là g(N). Thông thường, g(N) sẽ là một trong các hàm sau: 1, log N, N, N log N, N^2, N^3, hoặc 2^N.

Hiệu năng phụ thuộc vào đầu vào (Performance depends on inputs)

Chúng ta có thể đặc trưng hóa hiệu năng của một thuật toán thông qua trường hợp tốt nhất (best case), trường hợp xấu nhất (worst case), và trường hợp trung bình (average case).

Độ khó của một bài toán (Difficulty of a problem)

Để hiểu được độ khó của một bài toán cụ thể, chúng ta thường xem xét tốc độ tăng trưởng trong trường hợp xấu nhất của thuật toán tối ưu nhất có thể cho bài toán đó. Chúng ta có thể chặn trên (upper bound) độ khó của bài toán bằng hiệu suất của thuật toán tốt nhất từng được biết đến. Việc tìm ra một cận dưới (lower bound) tốt cho một bài toán thường là một việc rất khó.

Ký hiệu Big Oh (O), Big Omega (Ω), Big Theta (Θ)

Các ký hiệu này thường được sử dụng phổ biến trong lý thuyết thuật toán. Chúng có bản chất tương tự như ký hiệu ~ nhưng **bỏ qua các hệ số dẫn đầu (hằng số)**. Nhiều lập trình viên đang sử dụng ký hiệu Big-Oh một cách sai lệch khi cái họ thực sự muốn ám chỉ là tốc độ tăng trưởng (order of growth).

- Khi một người nói *"Thuật toán tìm kiếm nhị phân có độ phức tạp O(log N)"*, có thể hiểu theo những cách nào?
    - 1. Theo cách hiểu thông dụng : … *có thời gian chạy cỡ log N trong trường hợp tồi nhất*. Đây là cách hiểu và cách nói chấp nhận được trong cộng đồng lập trình, nhưng nên phát biểu chính xác phải là "*... có thời gian chạy logN trong trường hợp tồi nhất*", thì rõ ràng đỡ lẫn lộn.
    - 2. Chính xác toán học: … *có thời gian chạy là hàm có cận trên O(logN), có thể là hằng số, tuyến tính, logN, không xác định là loại nào*.
      Mỗi lần gặp tình huống này, bạn cần phân tích xem ý người nói là trường hợp nào trong hai trường hợp trên. Nên tốt nhất là nếu bạn là người nói, bạn nên nói chính xác luôn cho đỡ rủi ro hiểu nhầm.
- Khi một người nói *"Thuật toán tìm kiếm nhị phân có độ phức tạp O(log N) trong trường hợp trung bình"*, có thể hiểu theo những cách nào?
    - ý người ta là Θ(log N), chỉ là do thói quen mà dùng O
    - người ta không biết nó chính xác là loại gì, chỉ biết không vượt quá logN
- Trong bài thi/kiểm tra thì nên hiểu kiểu gì? Chúng ta quy ước với nhau là sẽ hiểu theo kiểu chuẩn, chính xác toán học, thay vì hiểu kiểu thói quen (quen mồm)

Tốc độ tăng trưởng trong trường hợp xấu nhất không phải là thông tin quyết định (Worst-case order of growth isn't everything)

Một thuật toán có tốc độ tăng trưởng lý thuyết tốt hơn thuật toán khác không đồng nghĩa với việc nó sẽ chạy nhanh hơn trong thực tế. Chúng ta sẽ bắt gặp một số phản ví dụ đáng chú ý trong môn học, bao gồm việc so sánh giữa `quicksort` (sắp xếp nhanh) và `mergesort` (sắp xếp trộn).

Phân tích bộ nhớ (Memory analysis)

Bạn cần nắm vững cách tính toán lượng bộ nhớ tiêu thụ của một lớp (`class`) dựa trên mô hình bộ nhớ 64-bit được hướng dẫn trong sách giáo trình.

Phân tích lý thuyết và thực nghiệm (Theoretical and empirical analysis)

Các giả thuyết được tạo ra từ phân tích lý thuyết (hoặc từ các phỏng đoán như giả thuyết định luật lũy thừa của chúng ta) cần phải được xác thực bằng dữ liệu thực tế trước khi có thể hoàn toàn tin cậy.

1. COS226. s26. precept1

Hãy xác định số lần hàm op() được gọi theo đánh giá tiệm cận (asymptotic), dưới dạng một hàm số của n, sử dụng cả ký hiệu xấp xỉ (~ tilde) và ký hiệu Big Theta (Θ, tức là bậc tăng trưởng - order of growth).

1.

```
for (int i = 10; i < n + 5; i += 2)
    op();
```

2.

```
for (int i = 1; i <= n * n * n; i *= 2)
    op();
```

3.

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < 100; j++)
        op();
```

4.

```
for (int i = 0; i * i < n; i++)
    for (int j = 1; j < n; j *= 3)
        op();
```

5.

```
for (int i = 0; i < n; i++)
    for (int j = 1; j < n; j *= 2)
        op();
```

6.

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < 100; j++)
        for (int k = 0; k < n; k++)
            for (int l = k; l < n; l++)
                op();
```

2. **COS226, Midterm f25, 1c**

Hàm op() được gọi bao nhiêu lần (tính theo n)?

```
for (int i = 0; i < n*n; i++)
    for (int j = i+1; j < n*n; j++)
        for (int k = 1; k <= n*n; k = k*2) op();
```

○ $\sim \frac{1}{2}\, n^2 \log_2 n$ ○ $\sim 2n^4$ ○ $\sim \frac{1}{2}\, n^4 \log_2 n$ ○ $\sim n^4 \log_2 n$ ○ $\sim 2n^4 \log_2 n$

3. **COS226, Midterm f25, 1d**

Biểu thức nào mô tả đúng về hàm $f(n) = 2n^2 + 2n + 6 \log_2 n$?

$O(\log n)$	$O(n)$	$O(n^2)$	$O(n^3)$
$\Omega(\log n)$	$\Omega(n)$	$\Omega(n^2)$	$\Omega(n^3)$

4. **COS226, Midterm s25, 1**

Xét một cây nhị phân tìm kiếm được định nghĩa bởi cài đặt Java sau đây:

```
public class BST<Key extends Comparable<Key>, Value> {
    private Node root;              // the root of the red-black BST
    private int n;                  // number of key-value pairs
    ...

    private class Node {
        private Key key;            // the key
        private Value value;        // the associated value
        private Node parent;        // link to parent
        private Node left;          // link to left subtree
        private Node right;         // link to right subtree
        private int count;          // number of nodes in subtree
    }
}
```

Một đối tượng BST sử dụng bao nhiêu bộ nhớ (tính bằng byte) theo kích thước n (số cặp key–value chứa trong bảng)?

Tính tất cả bộ nhớ (bao gồm cả tham chiếu đối tượng) mà đối tượng BST được cấp phát, nhưng không tính bộ nhớ cho các đối tượng String (mà client cấp phát). Sử dụng mô hình chi phí bộ nhớ 64-bit; đơn giản hóa câu trả lời bằng ký hiệu xấp xỉ (~).

5. **COS226, Midterm f24, 2a**

Đoạn code dưới đây in từ hello bao nhiêu lần? Giả sử n chia hết cho 100.

```
for (int i = 1; i <= n; i++)
    for (int j = n; j >= i; j--)
        for (int k = 1; k <= n; k = k + n/100)
            System.out.println("hello");
```

~200n ~50n^2 ~100n^2 ~½ $n^2 \log_{100} n$ ~n^3/100 ~100n^3

6. **COS226, Midterm f24, 2b**

Chọn biểu thức mô tả đúng về hàm $nlog^2 n + 3n\sqrt{n} - 5n$

$\sim n\sqrt{n}$ $\Theta(nlog^2 n)$ $O(n^3)$ $O(n\sqrt{n})$ $O(n)$ $\Omega(log\ n)$

Độ phức tạp thuật toán

Các chương trình được nhắc đến trong các bài dưới đây có mã nguồn tại *https://algs4.cs.princeton.edu/code/*

1.4.1. Chứng minh rằng số các bộ ba phần tử có thể chọn được từ N phần tử được tính bằng công thức N(N-1)(N-2)/6. Gợi ý: dùng quy nạp toán học

1.4.2. Sửa chương trình ThreeSum để chạy đúng ngay cả khi các giá trị int lớn đến mức có thể gây tràn số.

1.4.3. Sửa DoublingTest để dùng StdDraw vẽ các đồ thị chuẩn và log-log giống như trong bài giảng, điều chỉnh tỷ lệ để đồ thị luôn luôn chiếm một vùng lớn trên cửa sổ chương trình.

1.4.5. Cho biết kết quả áp dụng phép xấp xỉ cho các biểu thức dưới đây

a. $N + 1$
b. $1 + 1/N$
c. $(1+ 1/N)(1 + 2/N)$
d. $2N^3 - 15N^2 + N$
e. $lg(2N)/lgN$
f. $lg(N^2 + 1)/lgN$
g. $N^{100}/2^N$

1.4.6 Cho biết bậc tăng của thời gian chạy (theo hàm của N) của mỗi đoạn code sau:

a.
```
int sum = 0;
for (int n = N; n > 0; n /= 2)
      for(int i = 0; i < n; i++) sum++;
```

b.
```
int sum = 0;
for (int i = 1; i < N; i *= 2)
      for (int j = 0; j < i; j++) sum++;
```

c.
```
int sum = 0;
for (int i = 1; i < N; i *= 2)
      for (int j = 0; j < N; j++) sum++;
```

1.4.7. Phân tích thuật toán duyệt toàn bộ ThreeSum theo mô hình chi phí dựa trên các phép toán số học và so sánh.

1.4.8. Viết một chương trình tính số cặp số có giá trị bằng nhau từ một file chứa các số int. Nếu thuật toán của bạn đang là bậc hai, hãy tìm cách sử dụng Arrays.sort() để có thuật toán NlogN. (Arrays.sort() là hàm sắp xếp mảng sử dụng thuật toán loại NlogN)

1.4.10. Sửa thuật toán tìm kiếm nhị phân để nó luôn trả về phần tử có chỉ số nhỏ nhất có giá trị bằng khóa tìm kiếm mà vẫn đảm bảo thời gian chạy logN.

1.4.12. Viết một chương trình lấy input là hai chuỗi số, mỗi chuỗi gồm N giá trị int đã sắp xếp tăng dần, in ra tất cả các giá trị xuất hiện ở cả hai chuỗi, theo thứ tự tăng dần. Thuật toán cần có thời gian chạy tỷ lệ thuận với N trong trường hợp tồi nhất.

1.4.13. Với giả thiết về kích thước các kiểu dữ liệu như trong bài giảng. Hãy tính lượng bộ nhớ cần thiết cho một đối tượng thuộc các kiểu dưới đây (xem code của thư viện algs4 hoặc thư viện chuẩn Java)

a. Accumulator
b. Transaction
c. FixedCapacityStackOfStrings với kích thước mảng C và N phần tử trong stack
d. Point2D
e. Interval1D
f. Interval2D
g. Double

Khó hơn (*ai nên đọc mục này? (1) những ai muốn điểm 10 thi cuối kì, nhớ là 10 chứ không phải 8.5 hay 9.0, và (2) những ai thích học môn này***)**

1.4.14. *4-sum*. Tìm thuật toán cho bài tổng-4

1.4.15. *Faster 3-sum*. Để khởi động, hãy tìm thuật toán và cài TwoSumFaster sử dụng một thuật toán tuyến tính để đếm số cặp có tổng bằng 0 đối với mảng đã được sắp xếp (thay vì dùng tìm kiếm nhị phân như trong thuật toán NlogN). Sau đó áp dụng ý tưởng này để phát triển một thuật toán bậc hai cho bài tổng-3

1.4.16. *Closest pair*. Viết một chương trình nhận input là một mảng a[] gồm N giá trị double, tìm một cặp gần nhất (closest pair) - là hai giá trị có hiệu không vượt quá hiệu của cặp số bất kì khác trong mảng. Hiệu ở đây tính bằng giá trị tuyệt đối. Thuật toán cần có thời gian chạy NlogN trong trường hợp tồi nhất.

1.4.17. *Farthest pair*. Viết một chương trình nhận input là một mảng a[] gồm N giá trị double, tìm một cặp xa nhất (Farthest pair) - là hai giá trị có hiệu không nhỏ hơn hiệu của cặp số bất kì khác trong mảng. Hiệu ở đây tính bằng giá trị tuyệt đối. Thuật toán cần có thời gian chạy tuyến tính trong trường hợp tồi nhất.

1.4.18. *Local minimum of array*. Viết một chương trình nhận input là một mảng a[] gồm N giá trị int phân biệt, tìm một cực tiểu địa phương (local minimum) là chỉ số i sao cho a[i] nhỏ hơn a[i-1] và < a[i+1]. Chương trình chỉ nên dùng ~2lgN phép so sánh trong trường hợp tồi nhất.
Gợi ý:

.

1.4.19. *Local minimum of an matrix*. Cho input là mảng hai chiều a kích thước NxN gồm các số int phân biệt. Hãy thiết kế một thuật toán có thời gian chạy tỷ lệ thuật với N để tìm cực tiểu địa phương: một cặp chỉ số i và j sao cho a[i][j] nhỏ hơn cả bốn ô kề cạnh trên dưới trái phải. Thời gian chạy cần tỷ lệ thuận với N trong trường hợp xấu nhất.

1.4.20. *Bitonic search*. Một chuỗi số được coi là bitonic nếu nó gồm một chuỗi số nguyên tăng và liền sau là một chuỗi số nguyên giảm. Hãy viết một chương trình lấy input là một chuỗi bitonic gồm các số nguyên phân biệt và xác định xem trong chuỗi có xuất hiện một giá trị cho trước hay không. Chương trình cần sử dụng ~3lgN phép so sánh trong trường hợp tồi nhất.

1.4.22. *Binary search with only addition and subtraction*. Viết một chương trình nhận input là một mảng gồm các giá trị phân biệt kiểu int sắp xếp giảm dần và xác định xem mảng có chưa một giá trị int cho trước hay không. Bạn chỉ được dùng các phép cộng, trừ và một lượng bộ nhớ phụ là hằng số. Thời gian chạy của chương trình cần tỷ lệ thuận với log N trong trường hợp tồi nhất.
Gợi ý:

.

1.4.23. *Binary search for a fraction*. Để tìm một phân số p/q sao cho 0 < p < q < N. Hãy thiết kế một hàm sử dụng các câu hỏi dạng “số đó có nhỏ hơn x không?” và sử dụng số câu hỏi là hàm loại logN.
Gợi ý:

1.4.24. *Thả trứng*. Giả sử bạn có một tòa nhà N tầng và nhiều trứng. Giả sử rằng trứng sẽ vỡ nếu bị thả xuống từ tầng F hoặc cao hơn, và sẽ không bị vỡ nếu thả từ tầng thấp hơn. Đầu tiên, hãy tìm chiến thuật xác định giá trị F sao cho số trứng vỡ là ~lgN khi dùng đến ~lgN lần thả trứng. Sau đó, tìm chiến lược mà chi phí giảm xuống còn ~2lgF.

1.4.25. *Thả hai trứng*. Xét câu hỏi trước, nhưng lần này giả sử bạn chỉ có 2 quả trứng, và chi phí của bạn được tính theo số lần thả trứng. Hãy tìm chiến lược xác định F sao cho số lần thả không vượt quá 2√N. Sau đó tìm chiến lược chỉ tốn chi phí c√F.

1.4.26. *3-collinearity*. Giả sử bạn có một thuật toán lấy input là N điểm phân biệt trên mặt phẳng và có thể trả về số bộ ba điểm nằm trên cùng một đường thẳng. Hãy chứng minh rằng bạn có thể dùng thuật toán này để giải bài 3-sum.

Gợi ý: Dùng đại số để chứng minh rằng các điểm (a, a^3), (b, b^3), và (c, c^3) là các điểm thẳng hàng khi và chỉ khi a + b + c = 0.

(các bài liên quan đến stack từ 1.4.27 đến 1.4.36, 1.4.37. chuyển sang bộ bài về stack và queue)

Thí nghiệm:
1.4.37. Autoboxing performance penalty. Hãy chạy thí nghiệm để xác định…..

1.4.38. Naive 3-sum implementation. Hãy chạy thí nghiệm để đánh giá cách cài đặt dưới đây của vòng lặp trong cùng của ThreeSum:

```
for (int i = 0; i < N; i++)
      for (int j = 0; j < N; j++)
            for (int k = 0; k < N; k++)
                  if (i < j && j < k)
                        if (a[i] + a[j] + a[k] == 0)
                              cnt++;
```

Cách làm: viết một phiên bản DoublingTest tính tỉ lệ so sánh thời gian chạy giữa cài đặt trên và ThreeSum (trong bài giảng).

1.4.39. Cải tiến độ chính xác cho doubling test. Hãy sửa DoublingRatio để lấy thêm tham số dòng lệnh thứ hai, tham số này quy định số lần cần gọi timeTrial() cho mỗi giá trị của N. Hãy chạy chương trình cho 10, 100, và 1000 lần thử và nhận xét về các kết quả.

1.4.40. 3-sum cho các giá trị ngẫu nhiên. Hãy xác lập và thẩm định một giả thuyết mô tả số các bộ ba số có tổng bằng 0 trong tập N giá trị int ngẫu nhiên. Nếu có thể, bạn hãy lập công thức toán, biết rằng các giá trị trong N có phân bố đều trong khoảng -M tới M, với M là số không nhỏ.

1.4.41. Thời gian chạy. Hãy ước tính thời gian chạy TwoSumFast, TwoSum, ThreeSumFast, và ThreeSum trên máy tính của bạn cho file dữ liệu chứa 1 triệu số. Hãy dùng chương trình DoublingRatio cho công việc này

1.4.42. Kích thước bài toán. Hãy ước tính giá trị lớn nhất của P mà bạn có thể chạy được TwoSumFast, TwoSum, ThreeSumFast, và ThreeSum trên máy tính của bạn cho file dữ liệu gồm 2^P nghìn số. Sử dụng DoublingRatio

1.4.44. Birthday problem. Viết một chương tình lấy số nguyên N từ tham số dòng lệnh và sử dụng StdRandom.uniform() để sinh ngẫu nhiên một chuỗi số nguyên trong khoảng từ 0 đến N-1. Hãy chạy thí nghiệm để thẩm định giả thuyết rằng số số nguyên được sinh ra trước khi chuỗi có cặp giá trị trùng nhau đầu tiên là ~$\sqrt{\pi N/2}$.

1.4.45. Coupon collector problem. Hãy sinh chuỗi ngẫu nhiên tương tự bài trên và chạy thí nghiệm để thẩm định giả thuyết rằng số số nguyên được sinh ra trước khi tất cả các giá trị có

thể (0..N-1) đã được sinh ra là ~NH_N.
Trong đó HN là số điều hòa thứ N (n-th harmonic number), có công thức như sau:
$H_N = 1 + ½ + ⅓ + … + 1/N$